The Deutsch-Jozsa Algorithm

The Deutsch-Jozsa Algorithm (1992)

- Solves a generalization of the problem solved by the Deutsch's algorithm, for functions acting on more than one qubit
- We are given an unknown function

$$f: \ \{0,1\}^n \to \{0,1\}$$

that is either constant or balanced

- A function f is balanced if it takes the value 0 on half of the inputs $x \in \{0,1\}^n$, and the value 1 for the other half

$$|\{x \in \{0,1\}^n \mid f(x) = 0\}| = |\{x \in \{0,1\}^n \mid f(x) = 1\}| = 2^{n-1}$$

THE DEUTSCH-JOZSA PROBLEM

INPUT: A black-box (oracle) for computing an unknown function

$$f: \ \{0,1\}^n \to \{0,1\}$$

that is either constant or balanced

PROBLEM: determine whether f is constant or balanced

Exact classical solution

- In the worst case we cannot decide with certainty whether f is constant or balanced with less than $2^{n-1}+1$ queries

  BEST CASE SCENARIO

  the first 2 queries have different answers ($\rightarrow$ f is balanced)

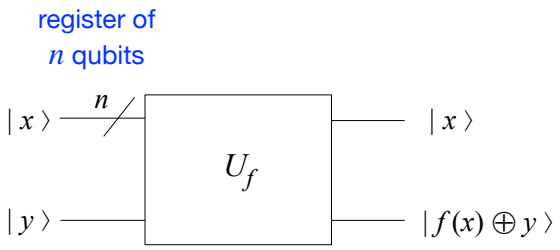
  WORST CASE SCENARIO

  same answer for 2^{n-1} queries - we need one more before telling whether f is constant or balanced

Quantum solution

- Takes advantage of quantum parallelism and quantum interference
- Determines whether f is constant or balanced making only 1 query to a quantum version of a reversible circuit for f

  $\rightarrow$ quantum oracle



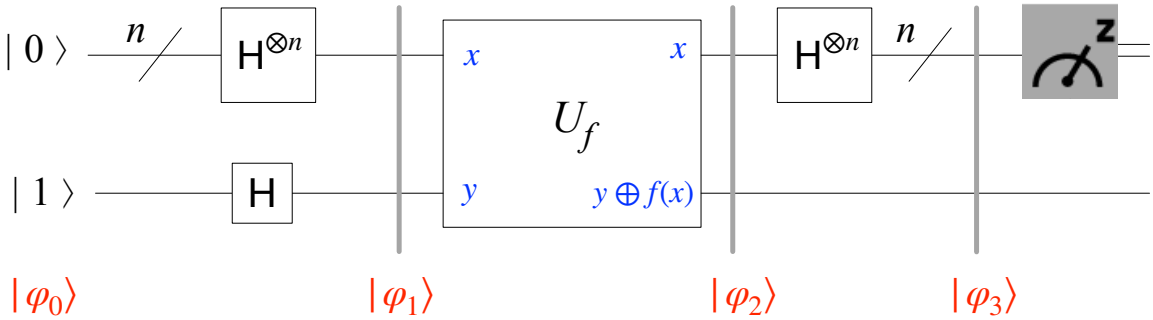
$|x\rangle = |\underbrace{x_0\, x_1\, \ldots\, x_{n-1}}_{n \text{ qubits}}\rangle$

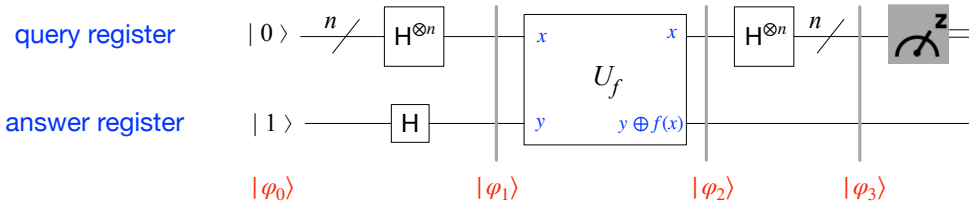
$|y\rangle$ one qubit

$f: \{0,1\}^n \rightarrow \{0,1\}$

$f(x)$ is one bit, so we can compute $f(x) \oplus y$

We will solve the problem by entering a superposition of all 2^n possible input states





$$|\varphi_0\rangle = |0\rangle^{\otimes n}\,|1\rangle$$

query register: n qubits, all prepared in the $|0\rangle$ state

answer register: $|1\rangle$

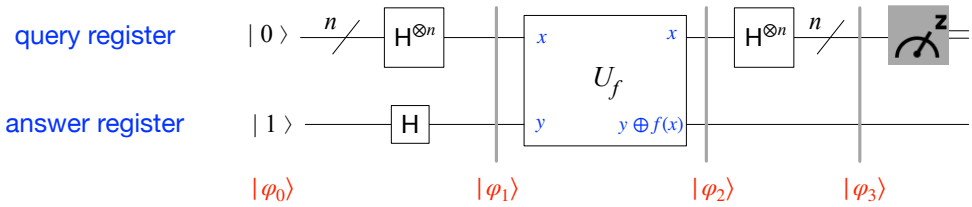
$$|\varphi_1\rangle = (H^{\otimes n} \otimes H)\,|\varphi_0\rangle = (H^{\otimes n} \otimes H)\,|0\rangle^{\otimes n}\,|1\rangle = \frac{1}{\sqrt{2^n}} \sum_{x\in\{0,1\}^n} |x\rangle\,|-\rangle$$

query register: uniform superposition of all configurations of n qubits

answer register: uniform superposition of $|0\rangle$ and $|1\rangle$

$$|\varphi_2\rangle = U_f\,|\varphi_1\rangle = \frac{1}{\sqrt{2^n}} U_f \left(\sum_{x\in\{0,1\}^n} |x\rangle\,|-\rangle \right) = \frac{1}{\sqrt{2^n}} \sum_{x\in\{0,1\}^n} (-1)^{f(x)} |x\rangle\,|-\rangle$$

The result of the query, $f(x)$, **is now stored in the amplitude** of the qubit superposition state



We now apply the Hadamard matrix $H^{\otimes n}$ on the query register to interfere all terms in the superposition

$$|\varphi_3\rangle = (H^{\otimes n} \otimes I)\,|\varphi_2\rangle = (H^{\otimes n} \otimes I)\,\frac{1}{\sqrt{2^n}} \sum_{x\in\{0,1\}^n} (-1)^{f(x)}\,|x\rangle\,|-\rangle$$

We need to understand the effect of $H^{\otimes n}$ on a computational basis state $|x\rangle = |x_0\,x_1\,\ldots x_{n-1}\rangle$

Effect of H on a single qubit $|x_i\rangle$

$$H\,|x_i\rangle = \frac{|0\rangle + (-1)^{x_i}\,|1\rangle}{\sqrt{2}} = \frac{1}{\sqrt{2}} \sum_{z_i\in\{0,1\}} (-1)^{x_i \cdot z_i}\,|z_i\rangle = \begin{cases} \dfrac{|0\rangle + |1\rangle}{\sqrt{2}} & x_i = 0 \\ \dfrac{|0\rangle - |1\rangle}{\sqrt{2}} & x_i = 1 \end{cases}$$

Effect of $H^{\otimes n}$ on $|x\rangle = |x_0\, x_1\, \ldots x_{n-1}\rangle$

Let $|x\rangle = |x_0\, x_1\, \ldots x_{n-1}\rangle$ denote a computational basis state on n qubits, i.e., $x \in \{0,1\}^n$ and each $x_i \in \{0,1\}$ denotes a particular bit value. Then

$$H^{\otimes n}|x\rangle = \frac{1}{\sqrt{2^n}} \sum_{z\in\{0,1\}^n} (-1)^{x\cdot z}\, |z\rangle$$

where the dot product $x \cdot z$ denote the inner product modulo two between $x, z \in \{0,1\}^n$

$$x \cdot z = x_0\, z_0 \oplus x_1\, z_1 \oplus \ldots \oplus x_{n-1}\, z_{n-1} \in \{0,1\}$$

$$H^{\otimes n}|x_0\, x_1\, \ldots x_{n-1}\rangle = H|x_0\rangle\, H|x_1\rangle\, \ldots\, H|x_{n-1}\rangle$$

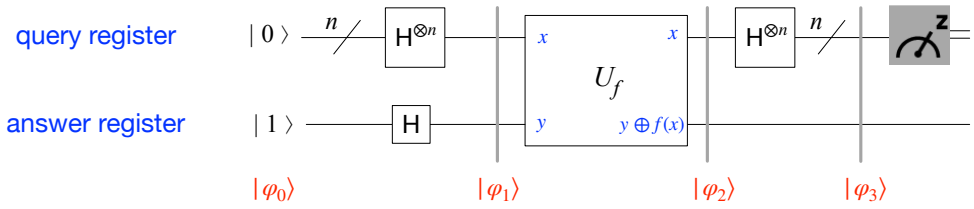
$$= \left(\frac{|0\rangle + (-1)^{x_0}|1\rangle}{\sqrt{2}}\right)\left(\frac{|0\rangle + (-1)^{x_1}|1\rangle}{\sqrt{2}}\right) \cdots \left(\frac{|0\rangle + (-1)^{x_{n-1}}|1\rangle}{\sqrt{2}}\right)$$

$$= \frac{1}{\sqrt{2^n}} \left(\sum_{z_0\in\{0,1\}} (-1)^{x_0\cdot z_0}|z_0\rangle\right)\left(\sum_{z_1\in\{0,1\}} (-1)^{x_1\cdot z_1}|z_i\rangle\right) \cdots \left(\sum_{z_{n-1}\in\{0,1\}} (-1)^{x_{n-1}\cdot z_{n-1}}|z_{n-1}\rangle\right)$$

$$= \frac{1}{\sqrt{2^n}} \sum_{z_0\in\{0,1\}} \sum_{z_1\in\{0,1\}} \cdots \sum_{z_{n-1}\in\{0,1\}} (-1)^{x_0\cdot z_0}\,(-1)^{x_1\cdot z_1} \ldots (-1)^{x_{n-1}\cdot z_{n-1}}\, |z_0\rangle|z_1\rangle \ldots |z_{n-1}\rangle$$

$$= \frac{1}{\sqrt{2^n}} \sum_{z\in\{0,1\}^n} (-1)^{x_0\cdot z_0 + x_1\cdot z_1 + \ldots + x_{n-1}\cdot z_{n-1}}\, |z\rangle = \frac{1}{\sqrt{2^n}} \sum_{z\in\{0,1\}^n} (-1)^{x_0\cdot z_0 \oplus x_1\cdot z_1 \oplus \ldots \oplus x_{n-1}\cdot z_{n-1}}\, |z\rangle$$

$$= \frac{1}{\sqrt{2^n}} \sum_{z\in\{0,1\}^n} (-1)^{x\cdot z}\, |z\rangle$$

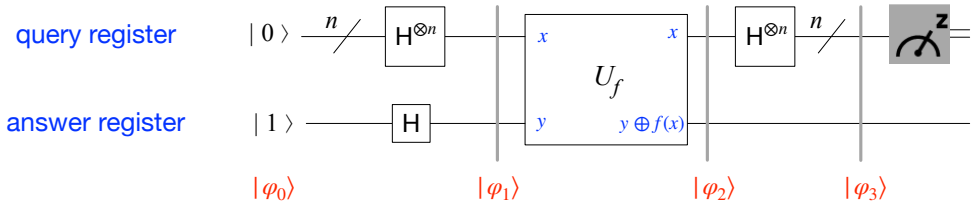


$$|\varphi_3\rangle = (H^{\otimes n} \otimes I)\,|\varphi_2\rangle = (H^{\otimes n} \otimes I)\, \frac{1}{\sqrt{2^n}} \sum_{x\in\{0,1\}^n} (-1)^{f(x)}|x\rangle\,|-\rangle$$

$$= H^{\otimes n}\left(\frac{1}{\sqrt{2^n}} \sum_{x\in\{0,1\}^n} (-1)^{f(x)}|x\rangle\right) I\,|-\rangle = \left(\frac{1}{\sqrt{2^n}} \sum_{x\in\{0,1\}^n} (-1)^{f(x)} H^{\otimes n}\,|x\rangle\right)|-\rangle$$

$$= \left(\frac{1}{\sqrt{2^n}} \sum_{x\in\{0,1\}^n} (-1)^{f(x)}\, \frac{1}{\sqrt{2^n}} \sum_{z\in\{0,1\}^n} (-1)^{x\cdot z}\,|z\rangle\right)|-\rangle$$

$$= \frac{1}{2^n} \sum_{z\in\{0,1\}^n} \left(\sum_{x\in\{0,1\}^n} (-1)^{f(x)+x\cdot z}\,|z\rangle\right)|-\rangle$$



$$|\varphi_3\rangle = \frac{1}{2^n} \sum_{z\in\{0,1\}^n} \left(\sum_{x\in\{0,1\}^n} (-1)^{f(x)+x\cdot z} \, |z\rangle \right) |-\rangle$$

Now, let us compute the amplitude of the state $|z\rangle = |0\,0\,\ldots\,0\rangle = |0\rangle^{\otimes n}$

$$\frac{1}{2^n} \sum_{x\in\{0,1\}^n} (-1)^{f(x)}$$

$$\frac{1}{2^n} \sum_{x\in\{0,1\}^n} (-1)^{1} = -1 \qquad f \text{ constant and equal to 1}$$

$$\frac{1}{2^n} \sum_{x\in\{0,1\}^n} (-1)^{0} = 1 \qquad f \text{ constant and equal to 0}$$

$$0 \qquad f \text{ balanced}$$

(positive and negative contributions cancel)

Amplitude of the state $|z\rangle = |0\,0\,\ldots\,0\rangle = |0\rangle^{\otimes n}$

$$\frac{1}{2^n} \sum_{x\in\{0,1\}^n} (-1)^{f(x)}$$

$$\frac{1}{2^n} \sum_{x\in\{0,1\}^n} (-1)^{-1} = -1 \qquad f \text{ constant and equal to 1}$$

$$\frac{1}{2^n} \sum_{x\in\{0,1\}^n} (-1)^{0} = 1 \qquad f \text{ constant and equal to 0}$$

$$0 \qquad f \text{ balanced}$$

- If f is **constant,** the amplitude of $|0\rangle^{\otimes n}$ is ± 1

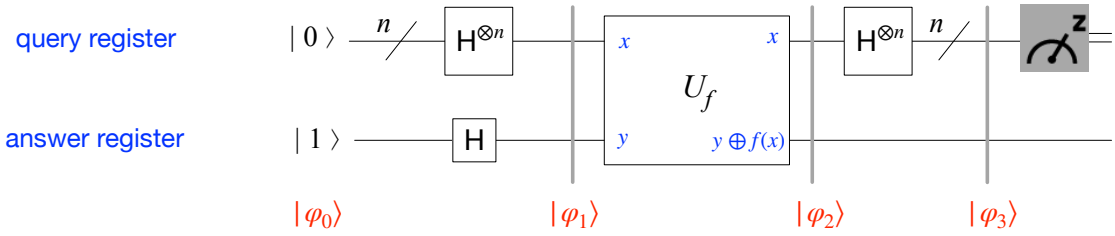
  and the amplitudes of all other states must be 0, since $|\varphi_3\rangle$ is a unitary vector

  ⇝ **A measurement is certain to return all 0s**, i.e., the binary string $000\ldots0$

- If f is **balanced,** positive and negative amplitudes cancel, and the overall amplitude of $|0\rangle^{\otimes n}$ becomes 0

  ⇝ **A measurement is certain NOT to return all 0s**

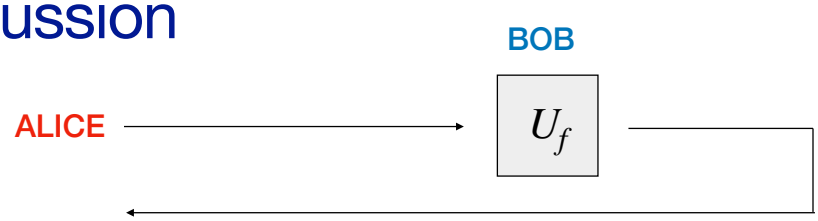
  we measure a result other than 0 on at least one qubit



To determine **with certainty** whether f is constant or balanced, the query register is measured:

- if the result of the measurement is all 0, then f is constant
- otherwise, if we measure at least one 1, f is balanced

Discussion



Imagine the oracle U_f is held by Bob, spatially separated from Alice, who is trying to determine whether f is constant or balanced

CLASSICALLY Alice transmits $2^{n-1}+1$ messages, each of size n bits
Bob replies to each message with one bit

QUANTUMLY The Deutsch-Jozsa algorithm requires only the transmission of a single message of $n+1$ qubits ($|0\rangle^{\otimes n}|1\rangle$) by Alice
Bob replies with a n qubits message

EXPONENTIAL REDUCTION IN THE AMOUNT OF INFORMATION TRANSFER REQUIRED TO SOLVE AN INSTANCE OF THE PROBLEM

- Deutsch's algorithm was the first algorithm that proved a quantum advantage: a reduction in query complexity compared to the classical one
- The Deusch-Jozsa algorithm generalizes Deutsch's algorithm, revealing the possibility of exponential speed-ups using quantum computers

HOWEVER

- the problem is not an important one
- it has no known applications
- using a probabilistic classical computer, it is possible to quickly determine with high probability whether f is constant or balanced

CLASSICAL RANDOMIZED ALGORITHM

→ **Repeat k times** (for k a parameter to be chosen as needed)

- pick $x \in \{0,1\}^n$ uniformly at random
- call U_f to evaluate $f(x)$
- if $f(x)$ differs from any previous query answer, halt and output **BALANCED**

→ Halt and output **CONSTANT**

ONE-SIDE ERROR ALGORITHM

The first output (BALANCED) is always correct

The second output (CONSTANT) can be wrong

Suppose f is balanced

- each random value $f(x)$ has probability 1/2 to be 0 or 1
- the probability that k random values are all 0 or all 1 is

$$2\frac{1}{2^k} = \frac{1}{2^{k-1}}$$

- thus the probability of declaring that a balanced function is constant is

$$\frac{1}{2^{k-1}}$$

- For any $\varepsilon > 0$, we can guarantee an error probability upper bounded by ε if the number of queries k is s.t.

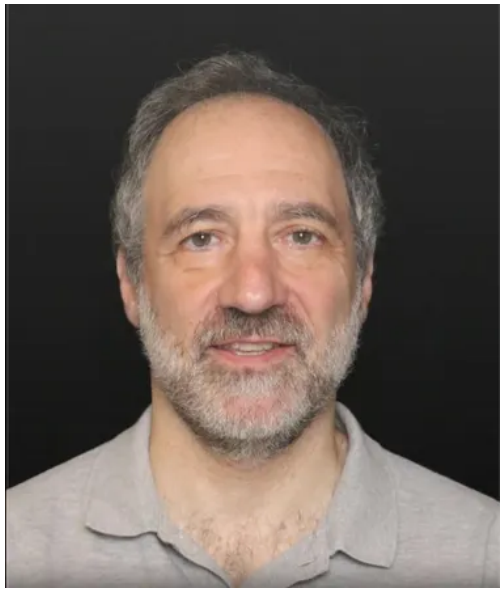
$$k \geq \left\lceil \log \frac{1}{\varepsilon} \right\rceil$$

- If we allow any level of error in the result, we lose the exponential separation between classical and quantum query complexity
- Exactly 0-error scenario in computation is unrealistic, we should always accept some (small) level of error
- There are other black-box problems for which an exponential separation exists between classical and quantum query complexity, even in presence of errors

$\Longrightarrow$ SIMON'S QUANTUM ALGORITHM (1994)

- Simon's research inspired Peter Shor, who then designed the polynomial time quantum algorithm for factorization

Simon's Quantum Algorithm



Simon's Algorithm (1994)

First quantum algorithm providing an exponential speed-up over any classical algorithm, including probabilistic ones

↝ It shows that **quantum computers are provably exponentially more efficient (in terms of number of queries) than bounded-error randomized classical algorithms**

It was the main inspiration for Shor's polynomial time quantum algorithm for factorization

SIMON'S PROBLEM

INPUT: A black-box (oracle) for computing a function

$$f: \ \{0,1\}^n \to \{0,1\}^n$$

with the property that there exists an unknown nonzero string $s \in \{0,1\}^n$ s.t.

$$\forall\, x, y \in \{0,1\}^n \quad f(x) = f(y) \iff y = x \ \text{ or } \ y = x \oplus s$$

PROBLEM: determine the secret string s by making queries to f (as few as possible)

$\oplus$ denotes bitwise addition module two (i.e., bitwise XOR)

We will treat the domain $\{0,1\}^n$ as the vector space $\mathbb{Z}_2^n$ over $\mathbb{Z}_2$, which consists of binary vectors of *n* components

The vector sum is the bitwise XOR, or modulo 2 sum $\oplus$

EXAMPLE

$n = 4$

$x \in \{0,1\}^4 = 1001$

$s \in \{0,1\}^4 = 1100$

$$y = x \oplus s = 0101 \in \{0,1\}^4$$

If $y = x \oplus s$, then $s = x \oplus y$ and $x = y \oplus s$

Functions f with Simon's property are **two-to-one functions**

- there are exactly two distinct inputs which produce the same output

$$f(x) = f(y) \iff y = x \oplus s$$

x and $y = x \oplus s$ produce the same output

x and $y = x \oplus s$ are different inputs since $s \neq 0^n$

A function f with Simon's property can be seen as a **periodic function**, with period s

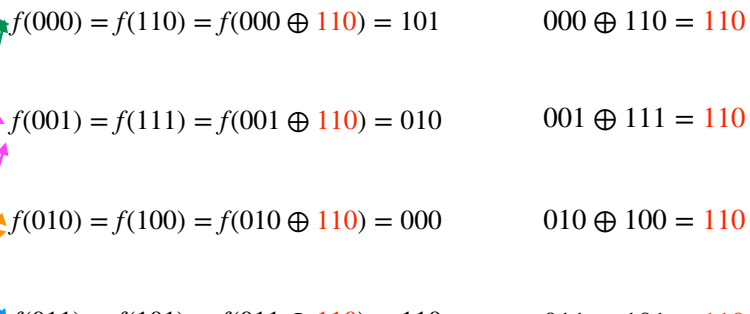
$$\forall x \in \{0,1\}^n, \quad f(x) = f(x \oplus s)$$

Simon's problem: given a function f, find its period s

Example

$n = 3$

x	$f(x)$
000	101
001	010
010	000
011	110
100	000
101	110
110	101
111	010



There are exactly two inputs which produce the same output.

These two inputs must be related:

by flipping the bits of one input at all locations where the secret string s has a 1, we obtain the other input

Classical query complexity

We can find the period s by finding a "collision pair" of inputs x and y such that

$$f(x) = f(y)$$

Indeed, the Simon's property implies

$$f(x) = f(y) \iff x = y \oplus s \iff s = x \oplus y$$

How many queries does it take to find a collision pair?

There is a classical randomized algorithm that solves Simon's problem with $O(\sqrt{2^n})$ queries

The algorithm is based on the "Birthday paradox"

An algorithm for Simon's problem

[Ronald de Wolf, *Quantum Computing: Lecture Notes*]

Classical randomized algorithm that solves Simon's problem using $O(\sqrt{2^n})$ queries

The algorithm makes t randomly chosen distinct queries

$$f(x_i), \quad x_i \in \{0,1\}^n, \quad 1 \le i \le t$$

If there is a collision among the queries, i.e., $f(x_i) = f(x_j)$ for some $x_i \neq x_j$ we output $s = x_i \oplus x_j$

In a sequence of t distinct queries there are

$$\binom{t}{2} = \frac{t(t-1)}{2} \approx \frac{t^2}{2}$$

pairs that could be a collision.

The probability for a fixed pair to form a collision is $\frac{1}{2^n - 1}$

since after one query on an input x_i, there is only one other input $x_j \neq x_i$ s.t. $f(x_i) = f(x_j)$

An algorithm for Simon's problem

By linearity of expectation, the expected number of collisions in our sequence will be roughly

$$\frac{\binom{t}{2}}{2^n - 1} \approx \frac{t^2}{2^{n+1}}$$

If we choose $t = \sqrt{2^{n+1}}$, we expect to have roughly one collision in our sequence, which is enough to find s

OBSERVATION

An *expected value* of 1 collision does not mean that we will have at least one collision *with high probability*, but a slightly more involved calculation shows that this latter statement is true

Lower bound

Simon proved that any classical randomized algorithm that finds s with high probability needs to make $\Omega(\sqrt{2^n})$ queries, so the above classical algorithm is essentially optimal.

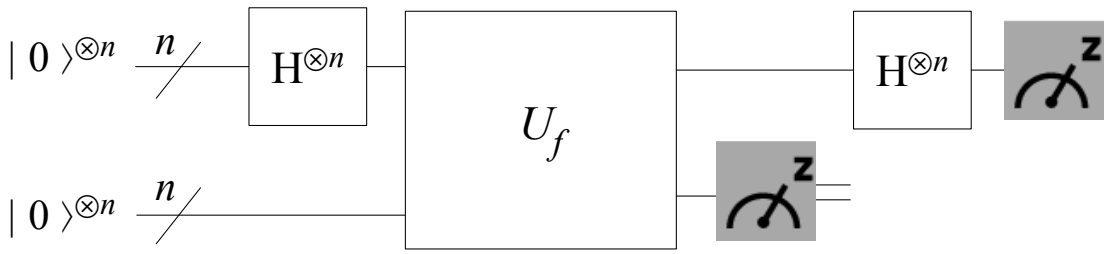
> **Simon's quantum algorithm solves this problem in polynomial time, performing an optimal O(n) queries**

> First proven **exponential separation between quantum algorithms and classical bounded-error algorithms** (with respect to query complexity)

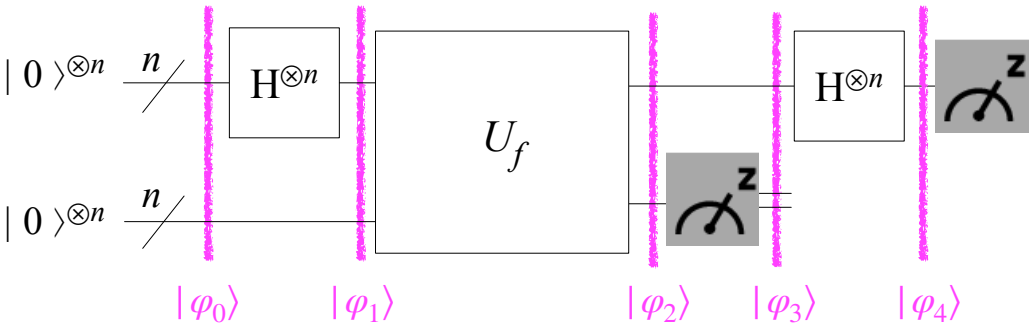
Simon's algorithm

Simon's algorithm is an hybrid quantum-classical algorithm, with two main steps

1. Run a quantum circuit $O(n)$ times



2. Perform a classical postprocessing on the output, efficiently in n, to determine the secret period s,



We start with two n-qubit registers, in the zero state

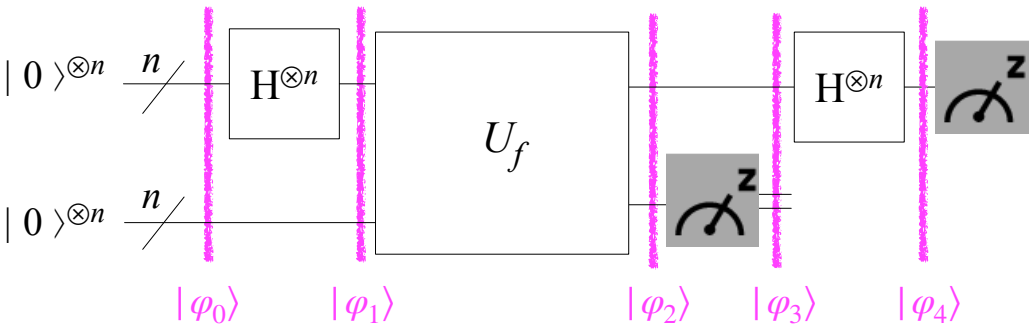
$$|\varphi_0\rangle = |0\rangle^{\otimes n}\,|0\rangle^{\otimes n}$$

We apply Hadamard gates on all qubits in the top register (Query register)

$$|\varphi_1\rangle = |+\rangle^{\otimes n}|0\rangle^{\otimes n} = \frac{1}{\sqrt{2^n}}\sum_{x\in\{0,1\}^n}|x\rangle\,|0\rangle^{\otimes n}$$

We then make a query, applying the oracle U_f

$$|\varphi_2\rangle = \frac{1}{\sqrt{2^n}}\sum_{x\in\{0,1\}^n}|x\rangle\,|f(x)\rangle$$



The second register is measured, and this measurement has an effect on the first register, since the two registers are entangled (this measurement is actually not necessary, but facilitates the analysis)

$$|\varphi_2\rangle = \frac{1}{\sqrt{2^n}}\sum_{x\in\{0,1\}^n}|x\rangle\,|f(x)\rangle \xrightarrow{I^{\otimes n}\otimes \text{measure}} |\varphi_3\rangle$$

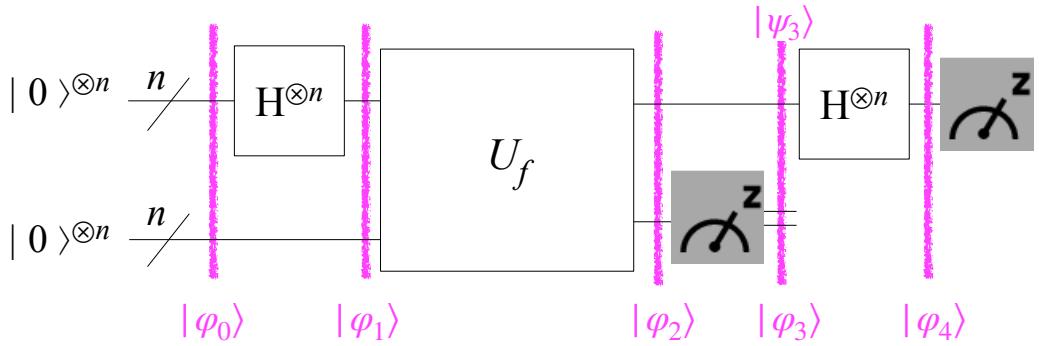
The measurement outcome will be particular value of the function, e.g., some value $w \in \{0,1\}^n$

Because of Simon's property, there are exactly two inputs that produce the output w, i.e.,

$$z \text{ and } z \oplus s \quad \text{such that} \quad f(z) = f(z \oplus s) = w$$

Thus, the state of the first register after measuring the second collapses to an equal superposition of the two inputs having value w

$$|\varphi_3\rangle = \left(\frac{1}{\sqrt{2}}|z\rangle + \frac{1}{\sqrt{2}}|z\oplus s\rangle\right)|w\rangle$$



We will now ignore the second register, and focus only on the first

$$|\varphi_3\rangle = \left(\frac{1}{\sqrt{2}}|z\rangle + \frac{1}{\sqrt{2}}|z \oplus s\rangle\right)|w\rangle \qquad |\psi_3\rangle = \frac{1}{\sqrt{2}}|z\rangle + \frac{1}{\sqrt{2}}|z \oplus s\rangle$$

The next step is to isolate the information about the period s that is stored in the first register. This can be done applying the Hadamard transform again

$$|\varphi_4\rangle = H^{\otimes n}|\psi_3\rangle = H^{\otimes n}\left(\frac{1}{\sqrt{2}}|z\rangle + \frac{1}{\sqrt{2}}|z \oplus s\rangle\right)$$

Recall that the Hadamard transform on a computational basis state $|x\rangle$ may be defined using the bitwise dot product as

$$H^{\otimes n}|x\rangle = \frac{1}{\sqrt{2^n}} \sum_{y \in \{0,1\}^n} (-1)^{x \cdot y}|y\rangle$$

where $x \cdot y = x_0 y_0 \oplus x_1 y_1 \oplus \ldots \oplus x_{n-1} y_{n-1} \in \{0,1\}$

$$|\varphi_4\rangle = H^{\otimes n}\left(\frac{1}{\sqrt{2}}|z\rangle + \frac{1}{\sqrt{2}}|z \oplus s\rangle\right)$$

$$= \frac{1}{\sqrt{2}}H^{\otimes n}|z\rangle + \frac{1}{\sqrt{2}}H^{\otimes n}|z \oplus s\rangle$$

$$= \frac{1}{\sqrt{2^{n+1}}}\left(\sum_{y \in \{0,1\}^n} (-1)^{z \cdot y}|y\rangle + \sum_{y \in \{0,1\}^n} (-1)^{(z \oplus s) \cdot y}|y\rangle\right)$$

$$= \frac{1}{\sqrt{2^{n+1}}} \sum_{y \in \{0,1\}^n} (-1)^{z \cdot y}(1 + (-1)^{s \cdot y})|y\rangle$$

Since

$$1 + (-1)^{s \cdot y} = \begin{cases} 2 & s \cdot y = 0 \\ 0 & s \cdot y = 1 \end{cases}$$

the state

$$|\varphi_4\rangle = \frac{1}{\sqrt{2^{n+1}}} \sum_{y \in \{0,1\}^n} (-1)^{z \cdot y} (1 + (-1)^{s \cdot y}) |y\rangle$$

is such that $|y\rangle$ has nonzero amplitude if and only if $s \cdot y = 0$ (modulo 2)

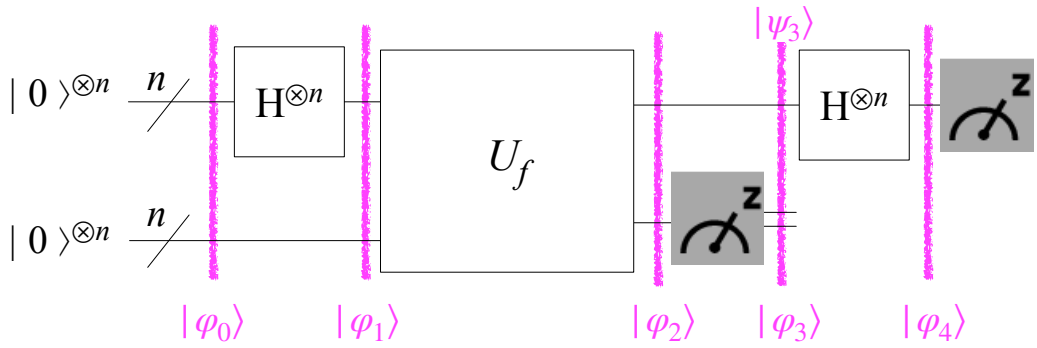
In particular,

- the amplitude associated to any value y such that $s \cdot y = 1$ is equal to 0
- the amplitude associated to any value y such that $s \cdot y = 0$ is equal to

$$\pm \frac{2}{\sqrt{2^{n+1}}} = \pm \frac{1}{\sqrt{2^{n-1}}}$$

Thus, $|\varphi_4\rangle$ is actually a **uniform superposition of computational basis states whose dot product with the unknown period s is 0**

$$|\varphi_4\rangle = \frac{1}{\sqrt{2^{n-1}}} \sum_{\substack{y \in \{0,1\}^n \\ s \cdot y = 0}} (-1)^{z \cdot y} |y\rangle$$



$$|\varphi_4\rangle = \frac{1}{\sqrt{2^{n-1}}} \sum_{\substack{y \in \{0,1\}^n \\ s \cdot y = 0}} (-1)^{z \cdot y} |y\rangle$$

The last step consists in a **measurement of the first register**

The outcome of such measurement is a uniformly random bit string $y \in \{0,1\}^n$, whose dot product (i.e., inner product modulo 2) with the period s is 0

This does not tell us s exactly, but it tells us "something about s":

a linear equation whose unknows are the bits of s

$$s \cdot y = 0 \quad \longrightarrow \quad s_0 y_0 \oplus s_1 y_1 \oplus \ldots \oplus s_{n-1} y_{n-1} = 0$$

We repeat Simon's algorithm until we have obtained a system of $n-1$ independent linear equations involving s, each corresponding to an outcome $y \in \{0,1\}^n$

Recall that y can be treated as a vector of the vector space $\mathbb{Z}_2^n$

The linear system can be solved efficiently by a classical algorithm, e.g., Gaussian elimination modulo 2

This is the **second step of Simon's algorithm**, the classical post-processing that can be implemented with a classical circuit of size roughly $O(n^3)$

The solutions of the linear system will be 0^n and s, the solution of Simon's problem

We must run the quantum circuit $O(n)$ times since we need to obtain $n-1$ linear independent equations to solve the system, that is, $n-1$ linear independent binary vectors of n components

- Suppose that you have obtained $k < n-1$ linear independent vectors
- The probability that the next run of the circuit produces a vector that is linear independent of the earlier ones is

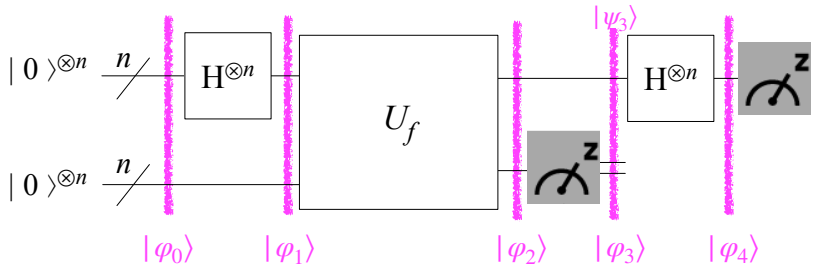
$$\frac{2^{n-1} - 2^k}{2^{n-1}} \geq 1/2$$

- Hence, an expected number of $O(n)$ runs suffices to find $n-1$ linear independent vectors

Example

$n = 3$

x	$f(x)$
000	101
001	010
010	000
011	110
100	000
101	110
110	101
111	010



$$|\varphi_0\rangle = |000\rangle\,|000\rangle$$

$$|\varphi_1\rangle = |+\rangle^{\otimes 3}|000\rangle = \frac{1}{2\sqrt{2}} \sum_{x\in\{0,1\}^3} |x\rangle|000\rangle$$

$$|\varphi_2\rangle = \frac{1}{2\sqrt{2}} \sum_{x\in\{0,1\}^3} |x\rangle|f(x)\rangle$$

The second register is measured, collapsing the first register to

$$|\psi_3\rangle = \frac{1}{\sqrt{2}}|z\rangle + \frac{1}{\sqrt{2}}|z\oplus s\rangle$$

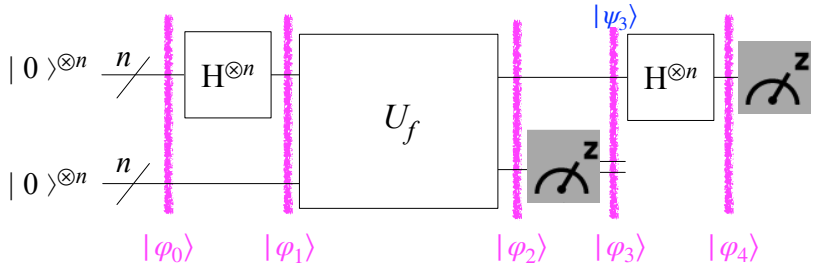
A second Hadamard transform is applied to the first register

$$|\varphi_4\rangle = \frac{1}{2} \sum_{\substack{y\in\{0,1\}^3 \\ s\cdot y = 0}} (-1)^{z\cdot y}|y\rangle$$

Example

$n = 3$

x	$f(x)$
000	101
001	010
010	000
011	110
100	000
101	110
110	101
111	010



$$|\varphi_4\rangle = \frac{1}{2} \sum_{\substack{y\in\{0,1\}^3 \\ s\cdot y = 0}} (-1)^{z\cdot y}|y\rangle$$

First measurement on the first register: 001

Second measurement on the first register: 111

$$\begin{cases} 0\,s_0 \oplus 0\,s_1 \oplus 1\,s_2 = 0 \\ 1\,s_0 \oplus 1\,s_1 \oplus 1\,s_2 = 0 \end{cases} \Longrightarrow \begin{cases} s_2 = 0 \\ s_0 \oplus s_1 \oplus s_2 = 0 \end{cases}$$

$$\Longrightarrow \begin{cases} s_2 = 0 \\ s_0 \oplus s_1 = 0 \end{cases}$$

The solutions are $s = 000$ and $s = 110$

Thus, the unique nonzero solution is the period $s = 110$

Appendix: The birthday paradox

- Phenomenon that in a group of just 23 people, there is actually a large probability (about a 50-50 chance) that at least two of them will have the same birthday, despite the fact that the number of possible birthday (365) is much larger than the number of people (23)

- **Intuitive explanation**:

  the number of **pairs** of people is actually quadratic in the number of people, and each pair has a 1/365 probability to have the same birthday (assuming birthdays are distributed uniformly random among people)

With 23 people we have **253** pairs

$$\binom{23}{2} = \frac{23 \cdot 22}{2} = 253$$

Assuming birthdays are distributed uniformly random among people, each pair has probability $\frac{1}{365}$ to have the same birthday

The chance of 2 people having different birthdays is

$$1 - \frac{1}{365} = \frac{364}{365} = 0.997260$$

The probability that all **253 pairs** have different birthdays can be approximated assuming that these events are independent, and hence by multiplying their probability together, which gives us

$$\left(\frac{364}{365}\right)^{253} = 0.4995 \quad (49.95\%)$$

(of course, different pairs may overlap and hence are not independent, but the idea works)
Then, the probability of someone sharing a birthday is

$$\approx 1 - \left(\frac{364}{365}\right)^{253} = 0.5005 \quad (50.05\%, \text{ just over half})$$

Birthday paradox: exact formula

To derive the exact formula, we compute the probability of each person having a different birthday, Assuming that birthdays are equally likely to happen on each of the 365 days of the year,

- The probability that Person 1 does not share his/her birthday with previously analyzed people is **1**
- The probability that Person 2 has a different birthday than Person 1 is **364/365**, as Person 2 may have any birthday other than the birthday of Person 1.
- Similarly, the probability that Person 3 has a different birthday than Person 1 and 2 is **363/365**, as Person 3 may have any birthday other than the birthdays of Persons 1 and 2
- This analysis continues until Person 23 is reached, whose probability of not sharing their birthday with people analyzed before , is **(365-22)/365 = 343/365**
- The probability of all 23 people having different birthdays is therefore

$$\frac{364 \cdot 363 \cdot \ldots \cdot 343}{365^{22}} = \frac{364!}{342!\,365^{22}} = 0.492703$$

- Since the event of at least two of the 23 persons having the same birthday is complementary to all *n* birthdays being different, its probability is

$$1 - 0.492703 = 0.507297 \quad (50.7297\%)$$